

## Assignment 4 / Option A

# Thesis Defense Game

**Student Name:** RenXuan

**Student ID:** ZY2557207

**Date:** 2026-06-06

**Hosted App:** [https://cicisteady.github.io/assignment2\\_blog/thesis\\_defense\\_game.html](https://cicisteady.github.io/assignment2_blog/thesis_defense_game.html)

## 1. Project Overview

---

For Assignment 4, I built a playable web game called **Defend Your Thesis**. The concept is a survival-defense game where the player is a graduate student defending a thesis core from academic threats such as bugs, deadlines, and peer reviewers.

## 2. Functional Software

---

The application runs in a browser and is integrated into the existing GitHub Pages blog.

- Character selection page with 7 playable characters
- Canvas-based maze and real-time rendering
- Thesis core defense mechanic
- Incoming Bugs, Deadlines, and Peer Reviewers
- Citation collectibles and score logic
- Keyboard controls with Arrow keys or WASD
- Mouse/touch movement by clicking or tapping the canvas
- Game timer, score, citation count, and thesis integrity HUD
- Victory overlay and replay flow
- Responsive layout for desktop and mobile screens

## 3. Game Design

---

The player moves through a maze, collects citations, and intercepts threats before they reach the thesis core. The round lasts 10 seconds, which keeps the game short and stable for

classroom demonstration.

Characters

| Character          | Role                               |
|--------------------|------------------------------------|
| Data Knight        | Balanced defender                  |
| Citation Mage      | Wider citation collection aura     |
| Method Engineer    | Strong and stable defender         |
| Coffee Sprinter    | Fastest movement                   |
| Algorithm Ninja    | Quick interception                 |
| Lab Guardian       | Durable thesis protection          |
| Reviewer Whisperer | Bonus score against peer reviewers |

4. Architecture

HTML page

-> shared site CSS

-> game JavaScript

-> Canvas game rendering

-> HUD and game state updates

The implementation uses `createState()` , `update(dt)` , `draw()` , and event listeners for controls.

5. AI-Assisted Development

AI helped plan the game framework, convert the assignment requirements into concrete features, and implement specific systems such as character selection, collision detection, score logic, and responsive UI.

AI was most useful when the request was specific, such as asking for a tile-based collision system or a Canvas rendering loop. The final code still required manual verification, link checking, and deployment integration.

## 6. Technical Details

---

- **Rendering:** HTML Canvas with a 28 by 21 tile maze.
- **Canvas Size:** 896 by 640 pixels with responsive CSS scaling.
- **Collision:** Future-position wall checks with X/Y axis movement.
- **Loop:** `requestAnimationFrame` with delta-time updates.
- **Controls:** Arrow keys, WASD, and pointer movement.

## 7. Verification

---

```
node --check site\assets\thesis_defense_game.js
```

The maze was checked as 21 rows by 28 columns, matching the 896 by 640 canvas when each tile is 32 pixels.

## 8. Reflection

---

This assignment showed the difference between an interesting prompt and a working software product. A stable game needs explicit state management, rendering, collision detection, controls, scoring, responsive layout, and deployment.

AI accelerated the implementation, but the developer still needed to define the goal, keep scope realistic, test the result, and prepare the final submission.

## 9. Conclusion

---

The final Assignment 4 submission is a functional hosted web game with character selection, gameplay, score logic, keyboard and mouse controls, and documentation explaining AI-assisted development.